

Comprehensive Documentation of C Projects with Code Highlights

Medjitna Brkacem/Lahmar Meriem Imene

May 18, 2025

Contents

1	Introduction	2
2	Basis and Core Concepts	2
2.1	Data Structures (ProjectFiles.c)	2
2.2	Input File Format	3
3	Definitions and Functionalities	3
3.1	ProjectFiles.c - Dictionary System	3
3.1.1	List Operations (TList)	3
3.1.2	Merged List Operations (BTList, CTList)	5
3.1.3	Stack Operations (TStack from BTList)	5
3.1.4	Queue Operations (TQueue from BTList)	6
3.1.5	Tree Operations (TTree)	7
3.2	ProjectFiles2.c - Utility and Test Functions	8
3.2.1	String Manipulation Functions	8
3.2.2	File and Word Operations	9
3.2.3	Word Property Functions	10
4	Error Handling and Known Issues	12
4.1	General Error Handling	12
4.2	Known Issues and Flaws (Primarily from ProjectFiles2.c)	12
4.3	Potential Unhandled Errors	12
5	How to Start and Setup	12
5.1	Prerequisites	12
5.2	Compilation	13
5.3	Input File: <code>file.txt</code>	13
5.4	Execution	13
6	Conclusion	13

1 Introduction

This document provides a comprehensive overview of two C projects. The first project, detailed in `ProjectFiles.c`, implements a dictionary-like system with functionalities for managing words, their synonyms, and antonyms using various data structures such as linked lists (singly, doubly, circular), stacks, queues, and binary search trees. The second project, found in `ProjectFiles2.c`, offers a collection of utility functions for string manipulation, file operations, and word property checks, along with a test menu for these functions.

The primary goal of `ProjectFiles.c` is to provide a robust system for dictionary operations, including loading data from a file, performing various searches and sorts, and manipulating data through different structural representations. `ProjectFiles2.c` serves as a testbed and utility library, demonstrating and testing individual algorithmic components, some of which are noted to have flaws in their original implementation.

2 Basis and Core Concepts

2.1 Data Structures (ProjectFiles.c)

The core of the dictionary project (`ProjectFiles.c`) revolves around several custom data structures designed to store and manage word data efficiently.

TList (Singly Linked List) A node in a singly linked list used for storing a word and an associated word (either a synonym or antonym, differentiated by an `indicator`). It also stores character counts (total, vowels, consonants).

```
1 struct TList {
2     char word[20];
3     struct TList* next;
4     char word2[20]; // Synonym or Antonym
5     int indicator; // 0 for synonym, 1 for antonym
6     int Chars;
7     int Consonants;
8     int Vowels;
9 };
10 typedef struct TList TList;
```

Listing 1: TList Structure Definition

BTList (Doubly Linked List) Used for merged word entries, storing a word, its synonym, and its antonym. It supports bidirectional traversal and also stores character counts.

```
1 struct BTList {
2     char word[20];
3     struct BTList* next;
4     char ant[20];
5     struct BTList* prev;
6     char syn[20];
7     int Chars;
8     int Consonants;
9     int Vowels;
10 };
11 typedef struct BTList BTList;
```

Listing 2: BTList Structure Definition

CTList (Circular Linked List) Similar to `BTList`, but implemented as a circular list, where the `next` pointer of the last element points to the head.

```
1 struct CTList {
2     char word[20];
3     struct CTList* next;
4     char ant[20];
5     int Chars;
6     char syn[20];
7     int Consonants;
8     int Vowels;
9 };
10 typedef struct CTList CTList;
```

Listing 3: CTList Structure Definition

TQueue (Queue) A queue structure holding `BTList` nodes, implemented with head and tail pointers.

```
1 struct TQueue {
2     BTList* head;
3     BTList* tail;
4 };
5 typedef struct TQueue TQueue;
```

Listing 4: TQueue Structure Definition

TStack (Stack) A stack structure holding `BTList` nodes, implemented with a top pointer.

```
1 struct TStack {
2     BTList* top;
3 };
4 typedef struct TStack TStack;
```

Listing 5: TStack Structure Definition

TTree (Binary Search Tree) Nodes for a binary search tree, where each node stores a word, its synonym, antonym, and character counts. The tree is typically ordered based on the number of characters in the word.

```
1 struct TTree {
2     char word[20];
3     struct TTree* right;
4     char ant[20];
5     struct TTree* left;
6     char syn[20];
7     int Chars;
8     int Consonants;
9     int Vowels;
10 };
11 typedef struct TTree TTree;
```

Listing 6: TTree Structure Definition

2.2 Input File Format

Both projects interact with an external file named `file.txt`. This file is expected to store dictionary entries, where each line represents a word, its synonym, and its antonym. The format parsed by functions like `getSynWords`, `getAntoWords` in `ProjectFiles.c` and file manipulation functions in `ProjectFiles2.c` is:

`word=synonym#antonym`

For example:

`happy=joyful#sad`

The system in `ProjectFiles.c` attempts to create `file.txt` if it does not exist upon startup.

3 Definitions and Functionalities

3.1 ProjectFiles.c - Dictionary System

This project provides a menu-driven interface to manage a dictionary.

3.1.1 List Operations (TList)

- **Creation & Loading:** Nodes are created dynamically. Key functions include `CreateNode()` for individual nodes, and `getSynWords()` / `getAntoWords()` for loading from file.

```

1 TList*CreateNode(){
2     TList *T=malloc(sizeof(TList));
3     if (T==NULL)
4     {
5         return NULL;
6     }
7     T->next=NULL;
8     T->word[0]='\0';
9     T->word2[0]='\0';
10    T->indicator=-1;
11    T->Chars=0;
12    T->Consonants=0;
13    T->Vowels=0;
14    return T;
15 }

```

Listing 7: TList Node Creation: CreateNode()

```

1 TList *getSynWords(FILE *file_name){
2     char buffer[100];
3     int checker;
4     TList* T=NULL,*Temp=NULL,*Tail=NULL;
5     while (fgets(buffer, sizeof(buffer), file_name)!=NULL)
6     {
7         char word[20];
8         char syn[20];
9         checker=sscanf(buffer, "%19[^\n]=%19[^\n]", word, syn);
10        if (checker==2)
11        {
12            // ... (node creation and population logic)
13        }
14        // ...
15    }
16    return T;
17 }

```

Listing 8: Loading Synonyms: getSynWords() (excerpt)

- **Display:** printList() displays the list.

```

1 void printList(TList* head) {
2     TList* current = head;
3     if (current == NULL) {
4         printf("List is empty.\n");
5         return;
6     }
7     printf("Contents of the list:\n");
8     head->indicator==0 ? DisplayHeaderSyn() : DisplayHeaderAnt();
9     while (current != NULL) {
10        if (current->indicator==0) {
11            printf(" %20s = %20s : Vowels %2d | Consonants %2d | Chars %2d \n",
12                current->word, current->word2, current->Vowels,
13                current->Consonants, current->Chars);
14        } else if (current->indicator==1) {
15            printf(" %20s # %20s : Vowels %2d | Consonants %2d | Chars %2d \n",
16                current->word, current->word2, current->Vowels,
17                current->Consonants, current->Chars);
18        }
19        current = current->next;
20    }
21 }

```

Listing 9: Displaying TList: printList() (excerpt)

- **Information Retrieval:** getInfWord()
- **Sorting:** sortWord() (alphabetical), sortWord2() (by char count), sortWord3() (by vowel count). These use a merge sort approach.

```

1 TList *sortWord(TList *syn,int size){
2     if(size<=1){
3         return syn;
4     }
5     int mid=size/2,counter=0;
6     TList* p1=syn,*p2=NULL,*t=NULL,*r1=NULL,*r2=NULL;
7     while(counter<mid-1){
8         p1=p1->next;
9         counter++;
10    }
11    p2=p1->next;
12    t=p1;
13    p1=p1->next; // p1 is now head of second half for recursive call
14    if (t!=NULL) { // If t is not NULL, it's the element before mid
15        t->next=NULL; // Split the list
16    } else { // Original list had 0 or 1 element, or error in logic
17        // This else block seems problematic if size > 1
18        // For size=2, mid=1, counter loops 0 times. p1 is head.
19        // p2=head->next. t=head. p1=head->next.
20        // head->next = NULL. This is correct for size=2.
21        // If size=1, mid=0, counter loops -1 times.
22        // For mid > 0.
23        // if (p1->next==NULL) { p2=NULL; } else { p2=p1->next; }

```

```

24     }
25     r1=sortWord(syn,mid); // Sort first half
26     r2=sortWord(p2,size-mid); // Sort second half
27     syn=merge2List1(r1,mid,r2,size-mid); // Merge sorted halves
28     return syn;
29 }

```

Listing 10: Sorting TList Alphabetically: sortWord() (Merge Sort)

- **Searching:** countWord() (by substring), palindromWord(), similarWord().
- **Modification:** updateWord() (updates in memory and file), addWord() (in-memory).
- **Utility:** countVowelsConsonants(), ListSize(), DestroyList().

3.1.2 Merged List Operations (BTList, CTList)

- **Merging:** merge() (to BTList), merge2() (to CTList).

```

BTList*merge(TList *syn, TList *ant){
1   if (syn==NULL||ant==NULL)
2   {
3       return NULL;
4   }
5   BTList*falseHead=BCreateNode(),*tail=falseHead;
6   while (syn!=NULL||ant!=NULL) // Should be syn!=NULL && ant!=NULL if they
7       are parallel
8   {
9       tail->next=BCreateNode();
10      tail->next->prev=tail;
11      strcpy(tail->next->word,syn->word);
12      strcpy(tail->next->syn,syn->word2);
13      strcpy(tail->next->ant,ant->word2);
14      tail->next->Vowels=syn->Vowels;
15      tail->next->Chars=syn->Chars;
16      tail->next->Consonants=syn->Consonants;
17      tail=tail->next;
18      ant=ant->next;
19      syn=syn->next;
20  }
21  // falseHead->next is the actual head. The falseHead node should be freed.
22  BTList* actualHead = falseHead->next;
23  if(actualHead) actualHead->prev = NULL; // First actual node's prev should
24      be NULL
25  free(falseHead);
26  return actualHead; // Corrected to return actualHead
}

```

Listing 11: Merging to Doubly Linked List: merge()

- **Display:** printBList(), printCList().
- **Destruction:** DestroyBList(), DestroyCList().

3.1.3 Stack Operations (TStack from BTList)

- **Creation:** createStack(), toStack() (transfers ownership).
- **Core Operations:** Push(), Pop(), IsEmptyStack(), Peek().

```

1 void Push(TStack*stk,BTList*New){
2     if (New==NULL||stk==NULL)
3     {
4         return;
5     }
6     if (stk->top==NULL)
7     {
8         stk->top=New;

```

```

9      New→next=NULL;
10     New→prev=NULL; // BTList node used in stack, prev might not be
                       strictly stack concept
11 }
12 else
13 {
14     New→next=stk→top;
15     New→prev=NULL;
16     stk→top→prev=New; // Making the old top point back to New
17     stk→top=New;
18 }
19 }

```

Listing 12: Push operation for TStack

- **Information Retrieval:** `getInfWordStack()`.
- **Sorting:** `sortWordStack()`.
- **Modification:** `deleteWordStack()`, `updateWordStack()`, `addWordStack()`.
- **Conversions:** `stackToQueue()` (copies nodes), `StacktoList()` (transfers ownership).
- **Utility:** `isPalindromeStack()`, `getSmallest()`, `StackRev()`.
- **Destruction:** `DestroyStack()`.

3.1.4 Queue Operations (TQueue from BTList)

- **Creation:** `createQueue()`, `toQueue()` (copies nodes).
- **Core Operations:** `EnQueue()`, `DeQueue()`.

```

1 void EnQueue(TQueue**Q,BTList*value){ // Should be TQueue* Q, not TQueue** Q
   for typical use
2   TQueue*T=*Q; // If Q is TQueue*, then TQueue* T = Q;
3   if (T==NULL) // Check if queue structure itself is NULL
4   {
5       *Q=createQueue(); // If Q is TQueue**, this allocates a new TQueue
6       T=*Q;
7       T→head=value; // value should be a new node if it's not to be shared
8       T→tail=value;
9       value→next = NULL; // Ensure new tail's next is NULL
10      value→prev = NULL; // Ensure new tail's prev is NULL (if head also)
11      return;
12  }
13  if (T→head == NULL) { // If queue exists but is empty
14      T→head = value;
15      T→tail = value;
16      value→next = NULL;
17      value→prev = NULL;
18      return;
19  }
20  // Add to tail
21  T→tail→next=value;
22  value→prev=T→tail; // BTList specific linking
23  T→tail=T→tail→next; // T→tail = value;
24  T→tail→next = NULL; // New tail's next must be NULL
25  return;
26 }

```

Listing 13: Enqueue operation for TQueue

- **Destruction:** `DestroyQueue()`.

3.1.5 Tree Operations (TTree)

- **Creation:** TCreateNode(), fillTree() (from file), toTree() (from stack).
- **Insertion:** AddNodeToTree().

```

1 void AddNodeToTree(TTree**root, TTree*New){
2     if (*root==NULL)
3     {
4         *root=New;
5         New->left = NULL; // Ensure children are NULL for new root
6         New->right = NULL;
7         return;
8     }
9     else
10    {
11        AddNodeToExistingTree(*root, New);
12        return;
13    }
14 }
15
16 void AddNodeToExistingTree(TTree*root, TTree*new_node){ // Renamed 'new' to
17     'new_node'
18     if (root==NULL) // Should not happen if called from AddNodeToTree's else
19     {
20         return ;
21     }
22     if (new_node->Chars >= root->Chars) // BST property based on Chars
23     {
24         if (root->right==NULL)
25         {
26             root->right=new_node;
27             new_node->left = NULL; // Ensure children are NULL
28             new_node->right = NULL;
29             return ;
30         }
31         else
32         {
33             AddNodeToExistingTree(root->right, new_node);
34         }
35     }
36     else // new_node->Chars < root->Chars
37     {
38         if (root->left==NULL)
39         {
40             root->left=new_node;
41             new_node->left = NULL; // Ensure children are NULL
42             new_node->right = NULL;
43             return ;
44         }
45         else
46         {
47             AddNodeToExistingTree(root->left, new_node);
48         }
49     }
50 }

```

Listing 14: Adding node to TTree

- **Display/Traversal:** printTreeInOrder(), printTreePreOrder(), printTreePostOrder().
- **Searching:** SerchInBst() (misspelled), getInfWordTree().
- **Deletion:** deleteWordBST().

```

1 TTree *deleteWordBST(TTree *tr, char *word) {
2     if (tr == NULL) {
3         return NULL;
4     }
5 }

```

```

6   int wordSize = strlen(word);
7   // Comparison for BST: typically based on a primary key, then secondary if
8   // needed.
9   // Here, it's Chars first, then the word string itself.
10  int comparisonChars = wordSize - tr->Chars;
11  int stringComparisonResult = strcmp(word, tr->word);
12
13  if (comparisonChars < 0 || (comparisonChars == 0 && stringComparisonResult
14  < 0) ) {
15      tr->left = deleteWordBST(tr->left, word);
16  } else if (comparisonChars > 0 || (comparisonChars == 0 &&
17  stringComparisonResult > 0) ) {
18      tr->right = deleteWordBST(tr->right, word);
19  } else { // Node to be deleted found (Chars match AND word matches)
20      // Case 1: Node with only one child or no child
21      if (tr->left == NULL) {
22          TTree *temp = tr->right;
23          free(tr);
24          return temp;
25      } else if (tr->right == NULL) {
26          TTree *temp = tr->left;
27          free(tr);
28          return temp;
29      }
30      // Case 2: Node with two children
31      // Get the in-order successor (smallest in the right subtree)
32      // NOTE: findSmallest in the code finds smallest by Chars, not
33      // necessarily in-order successor for general BST key.
34      // For a BST based on Chars, then word, this needs careful
35      // implementation.
36      // Assuming findSmallest(tr->right) correctly finds the in-order
37      // successor based on the tree's ordering.
38      TTree *inorderSuccessor = findSmallest(tr->right); // Need to ensure
39      // this finds the actual inorder successor
40
41      // Copy the content
42      strncpy(tr->word, inorderSuccessor->word, sizeof(tr->word) - 1);
43      tr->word[sizeof(tr->word) - 1] = '\0';
44      strncpy(tr->ant, inorderSuccessor->ant, sizeof(tr->ant) - 1);
45      tr->ant[sizeof(tr->ant) - 1] = '\0';
46      strncpy(tr->syn, inorderSuccessor->syn, sizeof(tr->syn) - 1);
47      tr->syn[sizeof(tr->syn) - 1] = '\0';
48      tr->Chars = inorderSuccessor->Chars;
49      tr->Vowels = inorderSuccessor->Vowels;
50      tr->Consonants = inorderSuccessor->Consonants;
51
52      // Delete the in-order successor node
53      tr->right = deleteWordBST(tr->right, inorderSuccessor->word);
54  }
55  return tr;
56 }

```

Listing 15: Deleting node from TTree: deleteWordBST() (excerpt)

- **Properties & Analysis:** isMirrorS(), height(), isBalencedBST() (misspelled), LowestCommonAncestor(), CountNodesRanges(), findSmallest().
- **Destruction:** DestroyTree().

3.2 ProjectFiles2.c - Utility and Test Functions

This project contains a set of standalone functions and a main function that provides a menu to test them.

3.2.1 String Manipulation Functions

- swapC(char* c, char* b): Swaps two characters.
- permutations(char str[], char *head): Attempts to print permutations. Marked as flawed.

```

1 void permutations(char str[], char *head){ // 'head' seems intended to be the
2     full original string
3     if (strlen(str)==2)
4     {
5         // Base case for string of length 2: print original and swapped
6         // Assuming 'head' points to the start of the modified (sub)string in
7         // the full buffer
8         printf("%s\n", head); // Prints current state
9         swapC(&str[0], &str[1]);
10        printf("%s\n", head); // Prints swapped state
11        swapC(&str[0], &str[1]); // Swap back
12    }
13    else
14    {
15        char temp_original_str[strlen(str)+1]; // Need +1 for null terminator
16        strcpy(temp_original_str, str);
17        for (int i = 0; i < strlen(str); i++) // Problem: should iterate
18            strlen(str) times if fixing one char
19        {
20            // Recursive call on substring str+1. This means 'head' in
21            // recursive calls
22            // still points to the original full string, but str points deeper.
23            permutations(str+1, head);
24            // Restore str for next iteration of this level
25            // This part is tricky. If str is modified by recursive calls,
26            // restoration is key.
27            // The current permutations() logic does not correctly isolate
28            // fixed characters and permute the rest.
29            // A common way is to swap str[0] with str[i], recurse on str+1,
30            // then swap back.
31            strcpy(str, temp_original_str); // Restore from copy
32            if (i < strlen(str) - 1) { // Avoid swapping with element beyond
33                current substring
34                swapC(&str[0], &str[i+1]); // Swaps first with i+1 for next
35                round of fixing first char
36            } else if (i == 0 && strlen(str) > 0) { // Ensure at least one
37                swap if loop runs once
38                // This specific swap logic for permutations needs careful
39                // review for correctness
40            }
41        }
42    }
43 }

```

Listing 16: Permutations function (flawed)

- printSub(char str[]): Prints subsequences. Noted to generate duplicates.

3.2.2 File and Word Operations

Operate on file.txt with format word=synonymantonym.

- countWordOccurence(FILE *file_name, char *word): Flawed count.
- DeleateWordInFile(FILE* original_file, char* word) (misspelled): Flawed deletion.

```

1 void DeleateWordInFile(FILE*original_file, char*word){ // original_file is
2     IGNORED
3     // ...
4     const char* filename = "file.txt"; // Hardcoded filename
5     FILE *temp_file;
6     // ...
7     original_file = fopen(filename, "r"); // Re-opens, shadowing parameter
8     // ... (Reads line by line, writes to temp_file if word not found in main
9     // ... (Replaces original with temp)
10 }

```

Listing 17: Delete Word In File (flawed)

- `removeWordOccurence(FILE *f, char *word)`: Depends on flawed functions.
- `updateWordInFile(const char* word, const char* syne, const char* anton)`: Updates entry in `file.txt`.

```

1 void updateWordInFile(const char* word, const char* syne, const char* anton){
2     if (word == NULL || syne == NULL || anton == NULL) {
3         return;
4     }
5     const char* filename = "file.txt";
6     FILE *original_file, *temp_file;
7     char buffer[256];
8     char temp_filename[] = "temp_dict.txt";
9     bool found_and_updated = false;
10
11     original_file = fopen(filename, "r");
12     if (original_file == NULL) { /* ... error handling ... */ return; }
13     temp_file = fopen(temp_filename, "w");
14     if (temp_file == NULL) { /* ... error handling ... */
15         fclose(original_file); return; }
16
17     while (fgets(buffer, sizeof(buffer), original_file) != NULL) {
18         char file_word[20], file_syn[20], file_ant[20];
19         char buffer_copy[sizeof(buffer)];
20         // ... (copy buffer, remove newline)
21         int scan_result = sscanf(buffer_copy, "%19[^\n]=%19[^\n]#%19[^\n]",
22                                 file_word, file_syn, file_ant);
23
24         if (scan_result >= 1 && strcmp(file_word, word) == 0) {
25             fprintf(temp_file, "%s=%s%s\n", file_word, syne, anton);
26             found_and_updated = true;
27         } else {
28             fprintf(temp_file, "%s", buffer);
29         }
30     }
31     fclose(original_file);
32     fclose(temp_file);
33
34     if (!found_and_updated) { /* ... handle not found ... */
35         remove(temp_filename); return; }
36     if (remove(filename) != 0) { /* ... error removing original ... */ }
37     else {
38         if (rename(temp_filename, filename) != 0) { /* ... error renaming temp
39             ... */ }
40         else { printf("File '%s' updated successfully.\n", filename); }
41     }
42 }

```

Listing 18: Update Word In File

- `updateWordItselfInFile(const char* word, const char* syne)`: Replaces main word.
- `countWordOccurenceInWord(FILE *file_name, char *word)`: Flawed count (main word only).
- `replaceWordOccurence(FILE *f, char *word, char *rep)`: Calls `updateWordItselfInFile`.

3.2.3 Word Property Functions

- `isPalindromWord(char *word, int size)` (misspelled): Flawed palindrome check.

```

1 bool isPalindromWord(char *word, int size){ // Misspelled: Palindrome
2     if (size==0||size==1) // Base case: empty or single char is palindrome
3     {
4         return true;
5     }
6     // Flaw: If word[0] != word[size-1], it's not a palindrome.
7     // This is correctly checked.
8     if (word[0] != word[size-1])
9     {

```

```

10     return false;
11 }
12 else // If they are equal, check the inner substring
13 {
14     // Recursive call: word+1 advances the start pointer.
15     // size-2 correctly reduces the size for the substring.
16     // The 'true &&' is redundant.
17     return /*true &&*/ isPalindromWord(word+1,size-2);
18     // The original code had 'return true&&isPalindromWord(word+1,size-2);'
19     // This recursive step is actually correct for checking.
20     // The issue might be in how it's called or if the definition of
21     // palindrome
22     // (e.g. case sensitivity) is implicitly different from the function's
23     // behavior.
24     // The main function's call: isPalindromWord(input_string,
25     // strlen(input_string))
26     // This seems like a standard recursive palindrome check.
27     // The warning in ProjectFiles2.c about "incorrect base cases and
28     // termination"
29     // might refer to a previous version or a misunderstanding, as this
30     // logic looks standard.
31     // However, potential issues could be off-by-one with size, or
32     // handling of non-alpha chars if any.
33 }
34 }

```

Listing 19: Palindrome Check (flawed recursion)

- `distinctSubseqWord(char *word)`: Calculates distinct subsequences. Has memory leak.

```

1 int distinctSubseqWord(char *word) {
2     if (word == NULL) return 0;
3     int n = strlen(word);
4     if (n == 0) return 1; // Empty string has one subsequence (itself)
5
6     // dp[i] will store the count of distinct subsequences of length i
7     int *dp = malloc((n + 1) * sizeof(int)); // MEMORY LEAK: dp is not freed
8     if (dp == NULL) return 0; // Malloc failure
9
10    // last[char] stores the last index of occurrence of char
11    int last[256]; // Assuming ASCII characters
12    for (int i = 0; i < 256; ++i) {
13        last[i] = 0; // Or -1 to indicate not seen
14    }
15
16    dp[0] = 1; // Empty prefix has one subsequence (empty string)
17
18    for (int i = 1; i <= n; ++i) {
19        dp[i] = 2 * dp[i - 1]; // Double the count from previous length
20
21        char current_char = word[i - 1]; // Character at current position
22        // (0-indexed)
23        if (last[(unsigned char)current_char] > 0) { // If char seen before
24            // Subtract subsequences ending with previous occurrence of
25            // current_char
26            dp[i] -= dp[last[(unsigned char)current_char] - 1];
27        }
28        // Ensure dp[i] doesn't go negative if using modulo arithmetic for
29        // large numbers
30        // (not apparently needed here as counts should be positive)
31
32        last[(unsigned char)current_char] = i; // Update last occurrence index
33        // (1-indexed for dp logic)
34    }
35
36    int result = dp[n];
37    // free(dp); // MISSING THIS LINE
38    return result;
39 }

```

Listing 20: Distinct Subsequences (memory leak)

4 Error Handling and Known Issues

4.1 General Error Handling

- **Memory Allocation:** Functions creating nodes (e.g., `CreateNode`, `BCreateNode`) check if `malloc` returns `NULL`.
- **File Operations:**
 - `ProjectFiles.c` `main` attempts to open/create `file.txt`.
 - Parser functions (`getSynWords`, `getAntoWords`) check `sscanf` results.
 - File update functions use temporary files and check for I/O errors.
- **Invalid Input:** Some functions check for `NULL` parameters. Menu systems handle basic `scanf` failures.

4.2 Known Issues and Flaws (Primarily from `ProjectFiles2.c`)

Comments in `ProjectFiles2.c` highlight:

- `permutations()`: Flawed head parameter usage.
- `printSub()`: Generates duplicate subsequences.
- `countWordOccurence()`, `countWordOccurenceInWord()`: Flawed recursive file I/O (no rewind).
- `DeleteWordInFile()`: Ignores `FILE*` parameter, hardcodes filename.
- `removeWordOccurence()`: Relies on flawed functions.
- `isPalindromWord()`: Warning about base cases/termination (though provided code looks standard).
- `distinctSubseqWord()`: Memory leak (`dp` array not freed).
- `main()` in `ProjectFiles2.c`: Test menu calls these flawed functions.

4.3 Potential Unhandled Errors

- **String Buffer Overflows:** Fixed-size arrays (e.g., `char word[20]`) risk overflow if input exceeds size. `sscanf` width specifiers (`%19s`) and `strncpy` mitigate this but require consistent application.
- **File Format Strictness:** Deviation from `word=synonymantonym` can cause parsing failures.
- **Recursive Depth:** Deep recursion (e.g., sorting/tree operations on huge datasets) could cause stack overflow.

5 How to Start and Setup

5.1 Prerequisites

- A C compiler (e.g., GCC, Clang).
- Standard C libraries.

5.2 Compilation

For `ProjectFiles.c`:

```
gcc ProjectFiles.c -o dictionary_system
```

Listing 21: Compilation Example for `ProjectFiles.c`

For `ProjectFiles2.c`:

```
gcc ProjectFiles2.c -o utility_tester
```

Listing 22: Compilation Example for `ProjectFiles2.c`

5.3 Input File: `file.txt`

- Both projects expect `file.txt` in the executable's directory.
- Format: `word=synonymantonym` per line.

Example:

```
happy=joyful#sad  
good=excellent#bad
```

- `dictionary_system` (from `ProjectFiles.c`) creates an empty `file.txt` if missing.

5.4 Execution

Dictionary System (`ProjectFiles.c`):

```
./dictionary_system
```

Listing 23: Running the Dictionary System

Launches a menu for dictionary functionalities.

Utility Tester (`ProjectFiles2.c`):

```
./utility_tester
```

Listing 24: Running the Utility Tester

Launches a menu to test individual functions (be mindful of documented flaws).

6 Conclusion

The C projects provide dictionary management (`ProjectFiles.c`) and utility functions (`ProjectFiles2.c`). `ProjectFiles.c` showcases data structure use for word operations. `ProjectFiles2.c` offers standalone functions, some with noted flaws. This documentation, with integrated code snippets, aims to clarify their structure and functionality. Future work could address issues and enhance features.